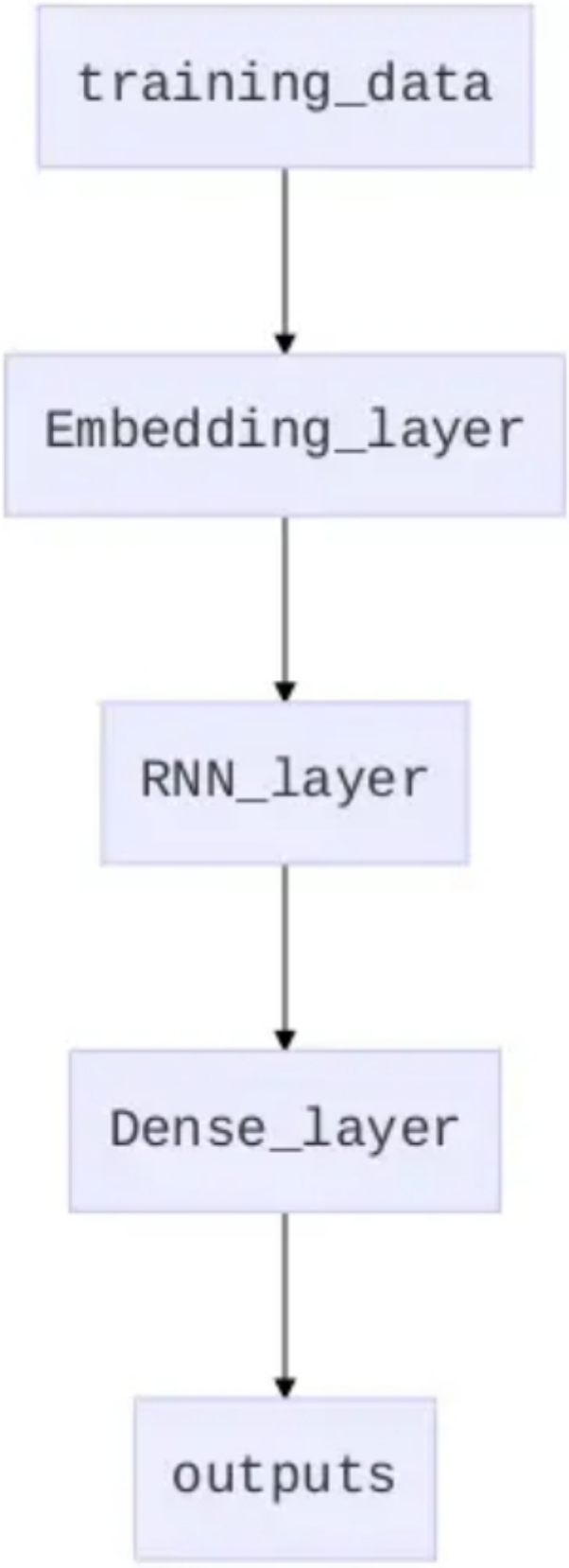
本课题尝试用IMDB数据集做电影评论分类
采用的预训练词向量
算法训练使用RNN模型

数据准备：IMDb电影评论数据集+谷歌的Word2vec模型

环境参数: Keras 2.1.2 + Tensorflow 1.4.1

算法流程：
主要基于embedding+RNN实现：


training_data
Embedding_layer
RNN_layer
Dense_layer
outputs

代码实现：

```
import os
import time
import copy
import gensim
import keras
import numpy as np
from sklearn.model_selection  import train_test_split
from keras.preprocessing.text import text_to_word_sequence,one_hot,Tokenizer
from keras.preprocessing.sequence import pad_sequences
from keras.models import Sequential,Model
from keras.layers import Input,Dense,GRU,LSTM,Activation,Dropout,Embedding
from keras.layers import Multiply,Concatenate,Dot


#************************1.读数据，获得X,Y************************
datapath = r'./data/train'
pos_files = os.listdir(datapath + './pos')
neg_files = os.listdir(datapath + './neg')
pos_all = []
neg_all = []

for pf,nf in zip(pos_files,neg_files):
        with open(datapath + '/pos' + '/' + pf,encoding = 'utf-8') as f:
                s = f.read()
                pos_all.append(s)
        with open(datapath + '/neg' + '/' + nf,encoding = 'utf-8') as f:
                s = f.read()
                neg_all.append(s)

X_orig = np.array(pos_all + neg_all)
Y_orig = np.array([1 for _ in range(12500)] + [0 for _ in range(12500)]) #1表示pos,0表示neg

#************************2.文本数值化************************
#设置词典大小为20000，文本序列最大长度为200
#统计词频，过滤掉没有代表性的低频词
#生成词向量
#直接把词向量输入给模型，无法进行继续调优(fine-tune)
#应该加载词向量的同时，继续进行fine-tune
vocab_size = 20000
maxlen     = 200
print("Start fitting the corpus......")
t = Tokenizer(vocab_size)
ti = time.time()
t.fit_on_texts(X_orig) #输入x,训练语料库得到统计信息
to = time.time()
word_index = t.word_index #不受vocab_size的影响
```

```
print('all_vocab_size',len(word_index))
print('Fitting time:',(to - ti),'s')
print("Start vectorizing the sentences......")
v_X = t.texts_to_sequences(X_orig)
print("Start padding......")
pad_X = pad_sequences(v_X,maxlen = maxlen,padding = 'post')
print("Finished")

#************************3.筛掉低频词，形成高频词字典************************
x = list(t.word_counts.items())
s = sorted(x,key = lambda p:p[1],reverse = True)      #按词频排序
small_word_index = copy.deepcopy(word_index)          #防止原来的字典也被改变了
print("Removing less freq words from word_index dict......")
for item in s[20000:]:
        small_word_index.pop(item[0])
print("Finished")
print(len(small_word_index))
print(len(word_index))

#************************4.词向量的导入************************
#导入GoogleNews预训练好的300维word2vec词向量
model_file = './word2vec_model/GoogleNews-vectors-negative300.bin'
print("Loading word2vec model......")
w2v_model  = gensim.models.KeyedVectors.load_word2vec_format(model_file,binary = True,limit=10
#把词向量，于word_index结合，构建一个embedding matrix
"""
先随机初始化一个embedding matrix，这里需要注意的是，我们的词汇量vocab_size虽然是20000，
但是训练的时候还是会碰到不少词不在词汇表里，也在词向量也查不到，那这些词怎么处理呢？
我们就需要单独给这些未知词（UNK）一个index，在keras的文本预处理中，会默认保留index=0给这些未知词。
"""
embedding_matrix = np.random.uniform(size = (vocab_size + 1,300))
#(vocab_size + 1) * 300
#vocab_size + 1:+1是要留一个给index = 0
#300:300维的词向量
print("Transfering to the embedding matrix......")
#sorted_small_index = sorted(list(small_word_index.items()),key = lambda x:x[1])
for word,index in small_word_index.items():
        try:
                #词向量与单词索引之间的映射
                word_vector             = w2v_model[word]
                embedding_matrix[index] = word_vector
        except:
                print("Word: [",word,"] not in word2vec_model ! Use random embedding instead.")

print("Finished")
print("Embedding matrix shape: \n",embedding_matrix.shape)
```

```
#************************5.划分训练集和测试集************************
np.random.seed = 1
random_indexs  = np.random.permutation(len(pad_X))
X = pad_X[random_indexs]
Y = Y_orig[random_indexs]
X_train,X_test,y_train,y_test = train_test_split(X,Y,test_size = 0.2)

#************************6.模型搭建************************
inputs = Input(shape = (maxlen,))
use_pretrained_w2v = True
if use_pretrained_w2v:
        w2v = Embedding(vocab_size + 1,300,input_length = maxlen,weights = [embedding_matrix])(
else:
        w2v = Embedding(vocab_size + 1,300,input_length = maxlen)(inputs)

h = LSTM(128)(w2v)
y = Dense(1,activation = 'sigmoid')(h)
m = Model(input = inputs,output = y)
m.summary()

m.compile(optimizer = 'adam',loss = 'binary_crossentropy',metrics = ['accuracy'])
m.fit(X_train,y_train,batch_size = 32,epochs = 3,validation_split = 0.15)
```

运行结果：

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_1 (InputLayer)         (None, 200)               0
_________________________________________________________________
embedding_1 (Embedding)      (None, 200, 300)          6000300
_________________________________________________________________
lstm_1 (LSTM)                (None, 128)               219648
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 129
=================================================================
Total params: 6,220,077
Trainable params: 6,220,077
Non-trainable params: 0
_________________________________________________________________
Train on 17000 samples, validate on 3000 samples
```

```
14496/17000 [=======================>.....] - ETA: 41s - loss: 0.4013 - acc: 0.8280
14528/17000 [=======================>.....] - ETA: 40s - loss: 0.4013 - acc: 0.8279
14560/17000 [=======================>.....] - ETA: 40s - loss: 0.4011 - acc: 0.8280
14592/17000 [=======================>.....] - ETA: 39s - loss: 0.4011 - acc: 0.8279
14624/17000 [=======================>.....] - ETA: 39s - loss: 0.4010 - acc: 0.8280
14656/17000 [=======================>.....] - ETA: 38s - loss: 0.4007 - acc: 0.8281
14688/17000 [=======================>.....] - ETA: 38s - loss: 0.4006 - acc: 0.8282
14720/17000 [=======================>.....] - ETA: 37s - loss: 0.4005 - acc: 0.8283
14752/17000 [=======================>....] - ETA: 37s - loss: 0.4003 - acc: 0.8285
14784/17000 [=======================>....] - ETA: 36s - loss: 0.4003 - acc: 0.8285
14816/17000 [=======================>....] - ETA: 36s - loss: 0.4004 - acc: 0.8284
14848/17000 [=======================>....] - ETA: 35s - loss: 0.4007 - acc: 0.8283
14880/17000 [=======================>....] - ETA: 35s - loss: 0.4002 - acc: 0.8286
14912/17000 [=======================>....] - ETA: 34s - loss: 0.3998 - acc: 0.8289
14944/17000 [=======================>....] - ETA: 34s - loss: 0.3994 - acc: 0.8292
14976/17000 [=======================>....] - ETA: 33s - loss: 0.3993 - acc: 0.8292
15008/17000 [=======================>....] - ETA: 32s - loss: 0.3993 - acc: 0.8292
15040/17000 [=======================>....] - ETA: 32s - loss: 0.3989 - acc: 0.8294
15072/17000 [=======================>....] - ETA: 31s - loss: 0.3986 - acc: 0.8296
15104/17000 [=======================>....] - ETA: 31s - loss: 0.3984 - acc: 0.8296
```